

Bridging the Vendor Gap:

Enabling AMD GPU Support for Awkward Array via ROCm/HIP
for the HL-LHC Era



Outline



HL-LHC requires heterogeneous, vendor-agnostic GPU computing.

the landscape

01 ScikitHEP tools historically rely on Nvidia/CUDA

CUDA-first kernels, CUDA-first CI, CUDA-first benchmarks — the rest of the ecosystem followed.

02 AMD-based supercomputers increasingly deployed

El Capitan, Frontier, LUMI, and the coming MI300A-class allocations put AMD GPUs in front of HEP workflows.

this work

New ROCm/HIP backend for Awkward Array



Supports nested, jagged, irregular HEP data on AMD GPUs



Benchmarked on MI210 / MI250 (CDNA2)



Forward-compatible with MI350 (CDNA4) and 288 GB HBM3E



Toward a truly hardware-agnostic HEP analysis stack

Motivation



HEP data is irregular: jagged, nested, variable-length.

irregular by design

Awkward Array is the Pythonic interface for this structure — ragged lists and nested records handled natively.

references

- M. Naumchyk

[Coffea schemas modifications](#)

GPU essential for HL-LHC

Luminosity increase at HL-LHC demands order-of-magnitude throughput gains — GPU acceleration is the only viable path.

references

- J. Letts

[WLCG Technical Evolution: Preparing for HL-LHC](#)

- J. Rembser

[Particle physics data analysis on the GPU](#)

- K.A. Mohrman

[GPU acceleration of end-user analyses at the LHC](#)

portability gap

CUDA ecosystem is mature; ROCm/HIP is catching up — but naïve ports lose 5–10× performance. Performance portability across vendors is unsolved.

references

- M. Naumchyk

[CUDA Acceleration of Awkward Array Using Python CCCL](#)

→ Need performance portability across vendors — rawkward's reason for being.

The awkward vendor gap

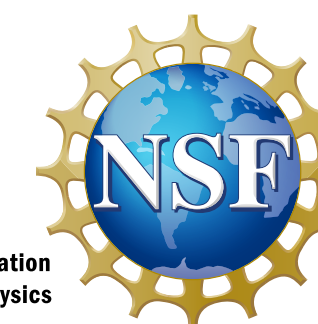
CUDA kernels exist and are optimized — HIP is a different story.

the gap

5–10×

slower on naïve HIP ports

Architectural differences matter for irregular kernels



our approach

HIP-specific optimization, not translation

Wavefront width, LDS layout, and occupancy differ fundamentally from CUDA. Each kernel needs tuning for the target arch — copy-paste ports leave most of the performance on the table.

Parity without changing the API

The same Rust API above, MI300A throughput below. rawkward kernels target the same performance envelope as their CUDA equivalents — measured, not assumed.

CUDA vs HIP: Architectural Mental Models



CUDA

warp = 32 threads

Warp width **32 threads**

Smaller warps mean more warps in flight — hides latency across more independent work.

Divergence **relatively cheap**

Divergent branches within a warp stall only the inactive lanes; the hardware is designed to handle this gracefully.

Scattered loads **scheduler-hidden**

Out-of-order memory schedulers tolerate irregular access patterns — naïve gather loops often still reach peak bandwidth.

HIP / AMD

wavefront = 64 lanes

Divergence **expensive**

64-wide wavefronts executing in lockstep lose half their throughput on any branch split — minimize control flow in hot paths.

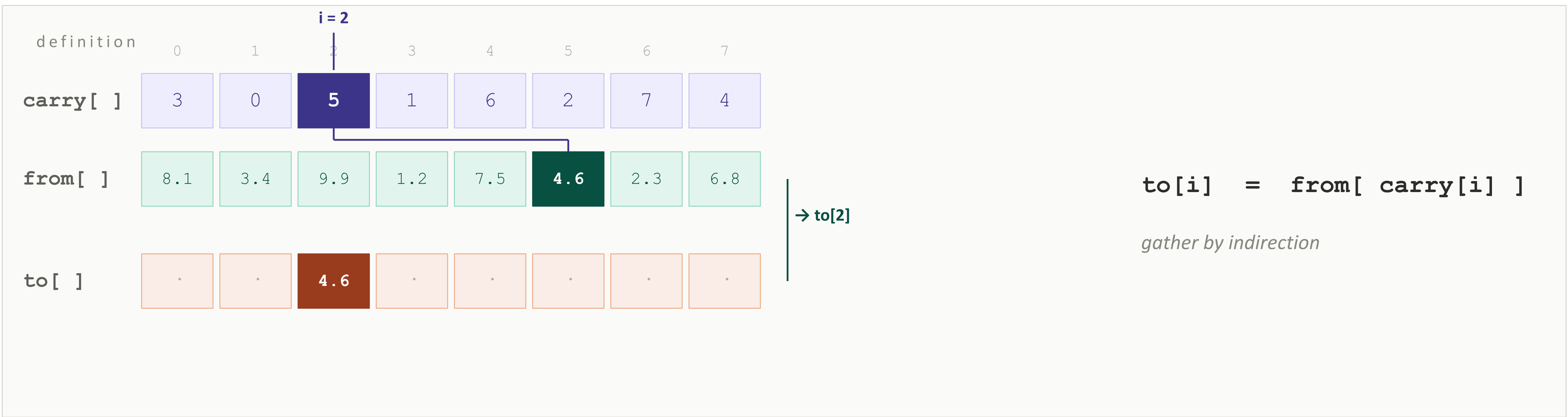
Register pressure **sensitive**

More lanes share the same register file; spills hit LDS or global memory hard. Occupancy tuning is critical.

Vectorization **critical**

Double-width SIMD and explicit LDS layout unlock full MI300A throughput — copy-paste CUDA ports leave most of this on the table.

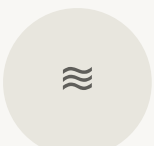
The Carry Kernel



properties



Core primitive for jagged indexing
Every nested-list operation reduces to a carry.



Memory-bound, irregular, divergence-sensitive
Random-access gather pattern defeats cache prefetchers; adjacent threads touch non-adjacent memory.

platform behavior

CUDA

✓ handles this well

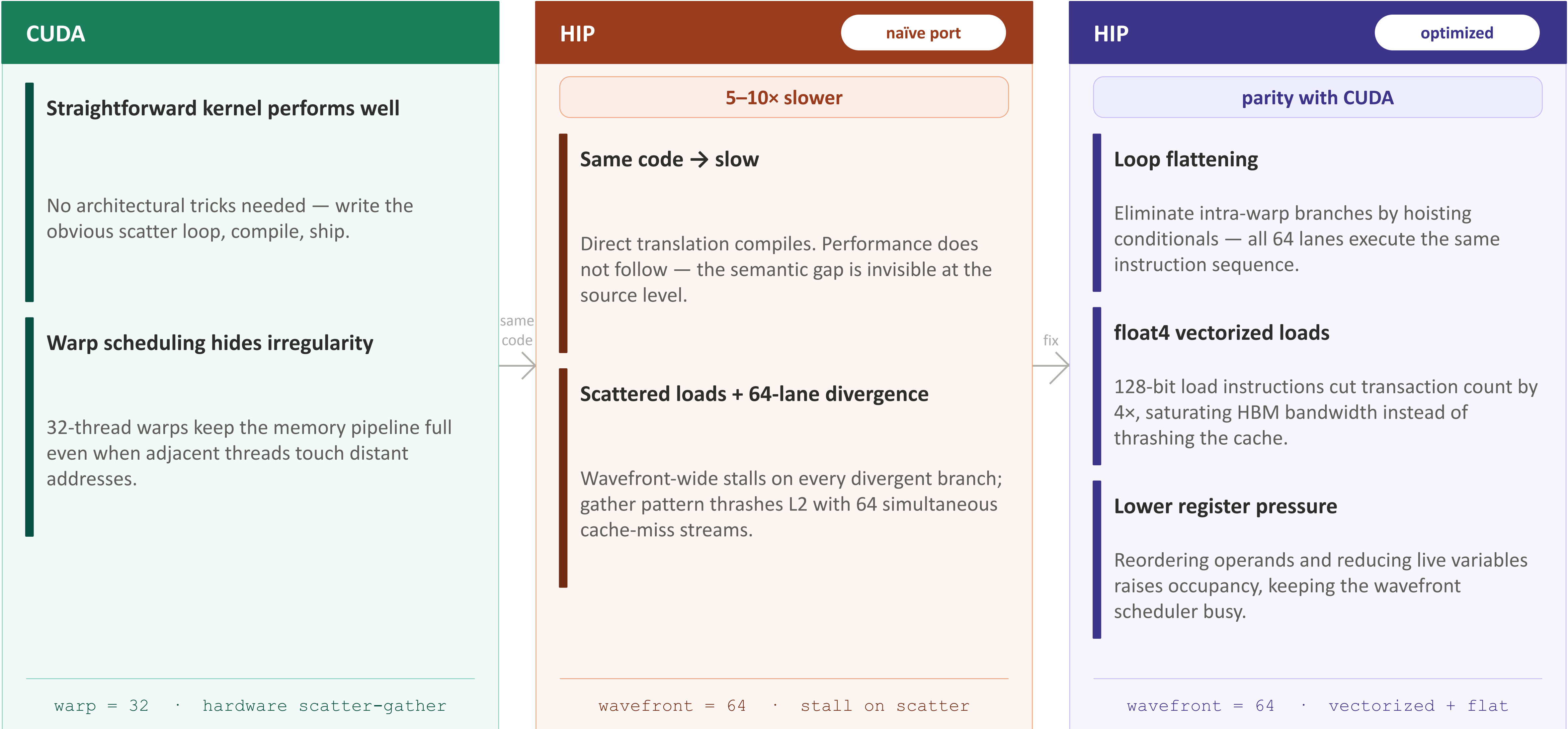
32-thread warps + hardware scatter-gather tolerate irregular access. Naïve port compiles, runs, and gets reasonable throughput.

HIP

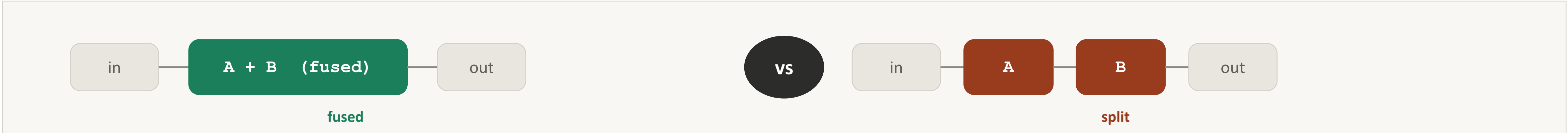
✓ needs loop flattening + vectorized loads

64-lane wavefronts amplify divergence. Flattening the loop to eliminate branches and packing loads into 128-bit vectors recovers the performance gap.

Carry Kernel: CUDA vs HIP



Fusion or Split: CUDA vs HIP



CUDA

fuse kernels

✓ fused kernels faster

Better L1 reuse

Intermediate values stay in registers or L1 between fused stages — no global memory round-trip, full bandwidth preserved.

Warp-friendly branching

32-thread warps handle branch divergence gracefully; fused control flow rarely stalls the pipeline enough to matter.

kernel fusion · fewer global round-trips

HIP

split kernels

✗ fused kernels slower

Divergence + register pressure

Fusing stages merges their register budgets across 64-lane wavefronts — occupancy drops, execution stalls multiply.

Split kernels outperform fused

Two clean, branch-free passes beat one fused pass: each kernel fits the wavefront register file and runs at full occupancy.

kernel split · occupancy over fusion



Rust as the Portable Orchestrator

C++ backend generation gets highly repetitive — a Rust macro/match pattern handles target-dependent dispatch cleanly.

the Rust API



Same Rust API for CUDA and HIP

Call sites are backend-agnostic — the same function signature compiles and dispatches correctly on both targets.



Backend decides fused vs split

The dispatch match arms select the right kernel strategy per vendor; the caller never sees that difference.



Rust enforces safety and correctness

Ownership, lifetime, and type rules catch data races and mismatched buffer sizes at compile time — not at 3 a.m.

```
macro_rules! dispatch {
  ($backend:expr, $fn:ident, $($arg:expr),*) => {
    match $backend {
      Backend::Hip => hip::$fn($($arg),*),
      Backend::Cuda => cuda::$fn($($arg),*),
    }
  }
}
```

HIP backend

HIP kernels optimized separately

Vectorized loads (float4), loop flattening, split kernel strategy — each technique targets the 64-lane wavefront architecture directly.

vectorization

loop flattening

CUDA backend

CUDA kernels fused for warp-friendly behavior

32-thread warp scheduler tolerates divergence; fusing stages keeps intermediates in registers and maximizes L1 hit rate.

kernel fusion

warp scheduling

Statistics-driven Microbenchmarking in Rust

Criterion.rs Benchmark Index

See individual benchmark pages below for more details.

- [reduce_sum_f32](#)

	cpu	metal
1024/128	reduce_sum_f32/cpu/1024/128	reduce_sum_f32/metal/1024/128
1048576/16384	reduce_sum_f32/cpu/1048576/16384	reduce_sum_f32/metal/1048576/16384
65536/4096	reduce_sum_f32/cpu/65536/4096	reduce_sum_f32/metal/65536/4096

- [reduce_sum_f32_roundtrip](#)

	metal	metal_alloc	metal_unified
1024/128	reduce_sum_f32_roundtrip/metal/1024/128	reduce_sum_f32_roundtrip/metal_alloc/1024/128	reduce_sum_f32_roundtrip/metal_unified/1024/128
1048576/16384	reduce_sum_f32_roundtrip/metal/1048576/16384	reduce_sum_f32_roundtrip/metal_alloc/1048576/16384	reduce_sum_f32_roundtrip/metal_unified/1048576/16384
65536/4096	reduce_sum_f32_roundtrip/metal/65536/4096	reduce_sum_f32_roundtrip/metal_alloc/65536/4096	reduce_sum_f32_roundtrip/metal_unified/65536/4096

- [reduce_sum_i64](#)

	cpu	metal
1024/128	reduce_sum_i64/cpu/1024/128	reduce_sum_i64/metal/1024/128
1048576/16384	reduce_sum_i64/cpu/1048576/16384	reduce_sum_i64/metal/1048576/16384
65536/4096	reduce_sum_i64/cpu/65536/4096	reduce_sum_i64/metal/65536/4096

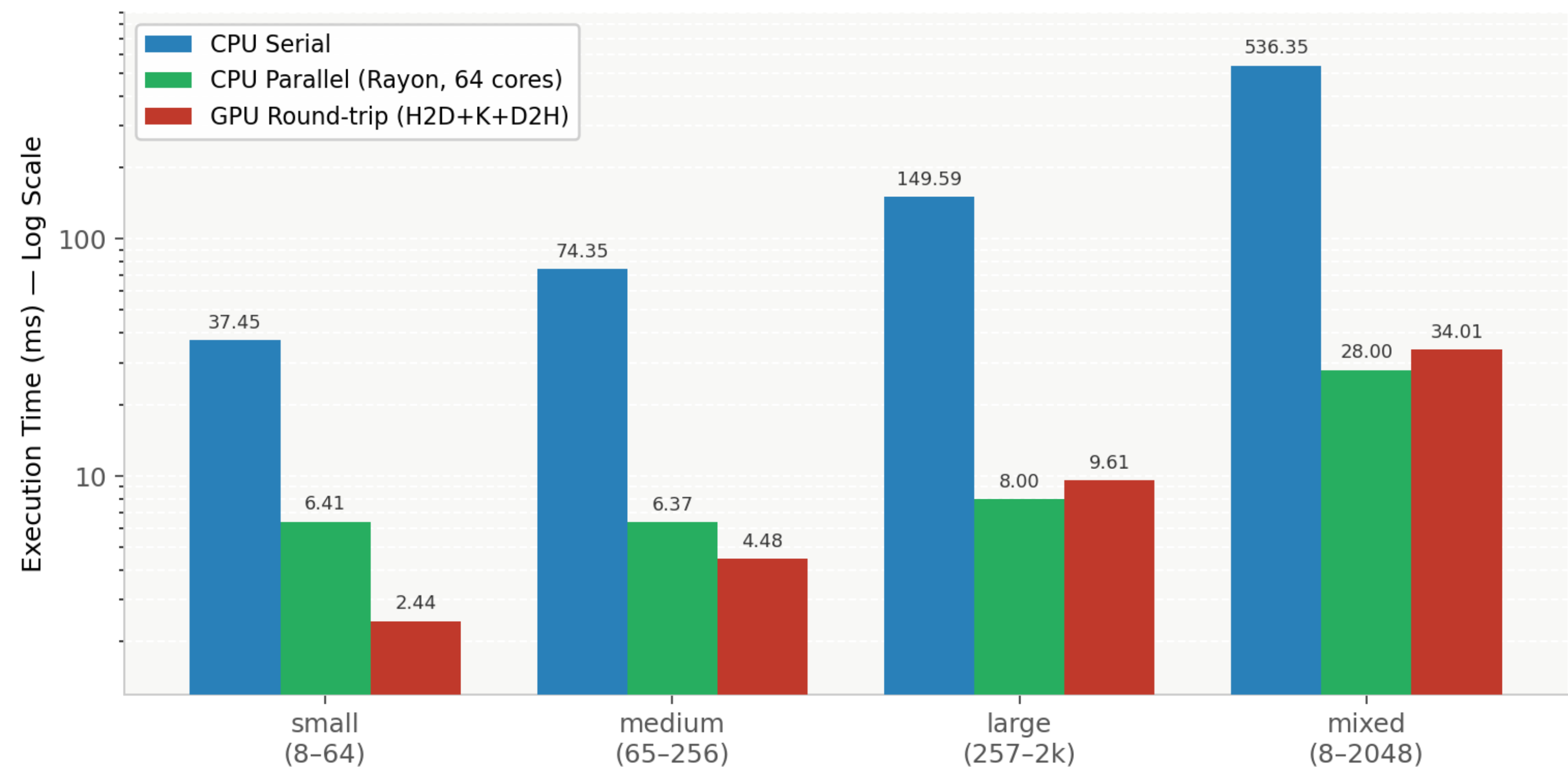
This report was generated by [Criterion.rs](#), a statistics-driven benchmarking library in Rust.

Benchmarks results are available at <https://github.com/ianna/CHEP2026-Bridging-the-Vendor-Gap-Enabling-AMD-GPU-Support-for-Awkward-Array>

Benchmarks: argsort kernel

GPU beats 128 CPU cores on small lists — even including the PCIe copy.

Performance Comparison: CPU vs GPU



dataset	cpu-ser (ms)	cpu-par (ms)	gpu-rnd (ms)	ser/gpu	par/gpu
small (8-64)	37.45	6.41	2.44	15.32×	2.62×
medium (65-256)	74.35	6.37	4.48	16.59×	1.42×
large (257-2k)	149.59	8.00	9.61	15.57×	0.83×
mixed (8-2048)	536.35	28.00	34.01	15.77×	0.82×

test node · della-milan

CPU · 2× AMD EPYC 7763

128 cores total (64/socket, no HT)
3.53 GHz boost · AVX2
L2 64 MiB · L3 512 MiB
8 NUMA nodes · ~1 TB DDR4
Milan / Zen 3 (glibc, ROCm host)

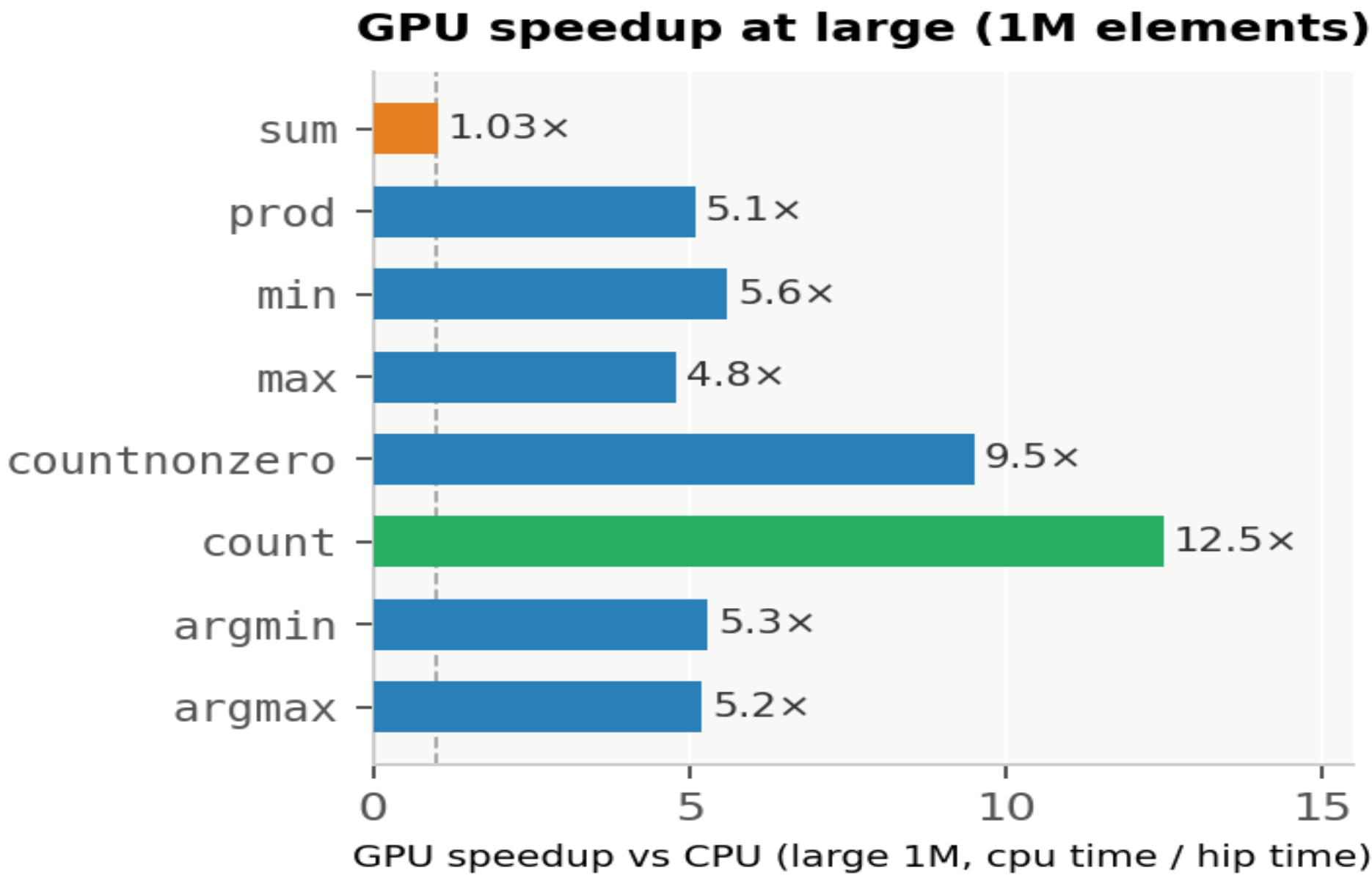
GPU · 2× AMD Instinct MI210

CDNA2 (gfx90a)
104 CUs · 6,656 shaders
64 GB HBM2e · ~1.6 TB/s
PCIe (discrete, no unified mem)
Nodes 8-9 in NUMA topology

argmin, argmax, count, countnonzero, max, min, prod, sum

Pattern: kernel-launch overhead ~36 μ s dominates at small sizes; GPU wins decisively at large.

kernel	small 1K (cpu / hip)	medium 64K (cpu / hip)	large 1M (cpu / hip)	GPU speedup (large)
argmax	608 ns / 36.4 μ s	39.6 μ s / 52.4 μ s	633 μ s / 120.5 μ s	5.2 $\times$
argmin	608 ns / 35.7 μ s	39.4 μ s / 52.3 μ s	633 μ s / 119.9 μ s	5.3 $\times$
count	1.02 μ s / 31.2 μ s	84.7 μ s / 45.0 μ s	1.04 ms / 83.3 μ s	12.5 $\times$
countnonzero	944 ns / 36.4 μ s	75.2 μ s / 51.7 μ s	1.14 ms / 119.9 μ s	9.5 $\times$
max	609 ns / 36.6 μ s	37.1 μ s / 52.4 μ s	575 μ s / 120.6 μ s	4.8 $\times$
min	683 ns / 36.2 μ s	42.1 μ s / 52.2 μ s	672 μ s / 120.6 μ s	5.6 $\times$
prod	598 ns / 37.0 μ s	35.8 μ s / 57.6 μ s	723 μ s / 141.2 μ s	5.1 $\times$
sum	330 ns / 36.6 μ s	13.2 μ s / 57.3 μ s	145 μ s / 140.8 μ s	1.03 $\times$



count gets 12.5 $\times$ — write-only pattern, no arithmetic, pure memory throughput. GPU issues 1 read per segment.

sum ties at 1.03 $\times$: AVX2 auto-vectorization on the EPYC hits full bandwidth, erasing the GPU edge at 1M.

crossover is between 1K and 64K — launch overhead (~36 μ s) dominates below, GPU wins cleanly above.

Profiling Summary

GPU launch floor

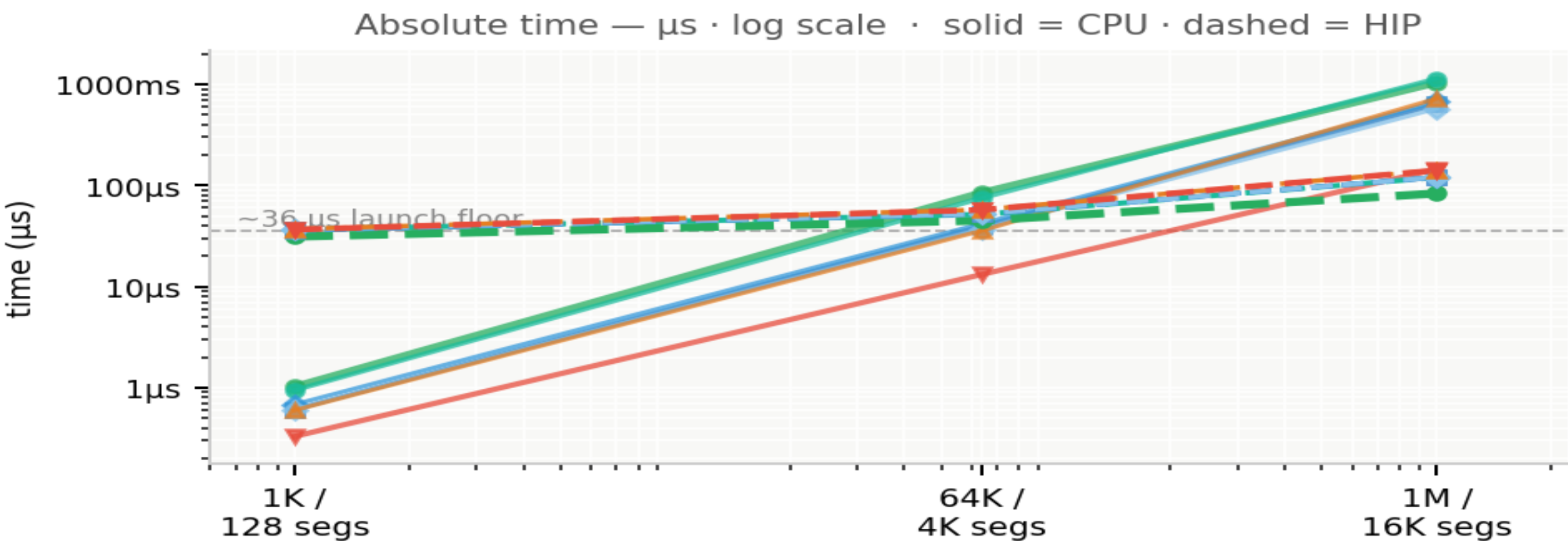
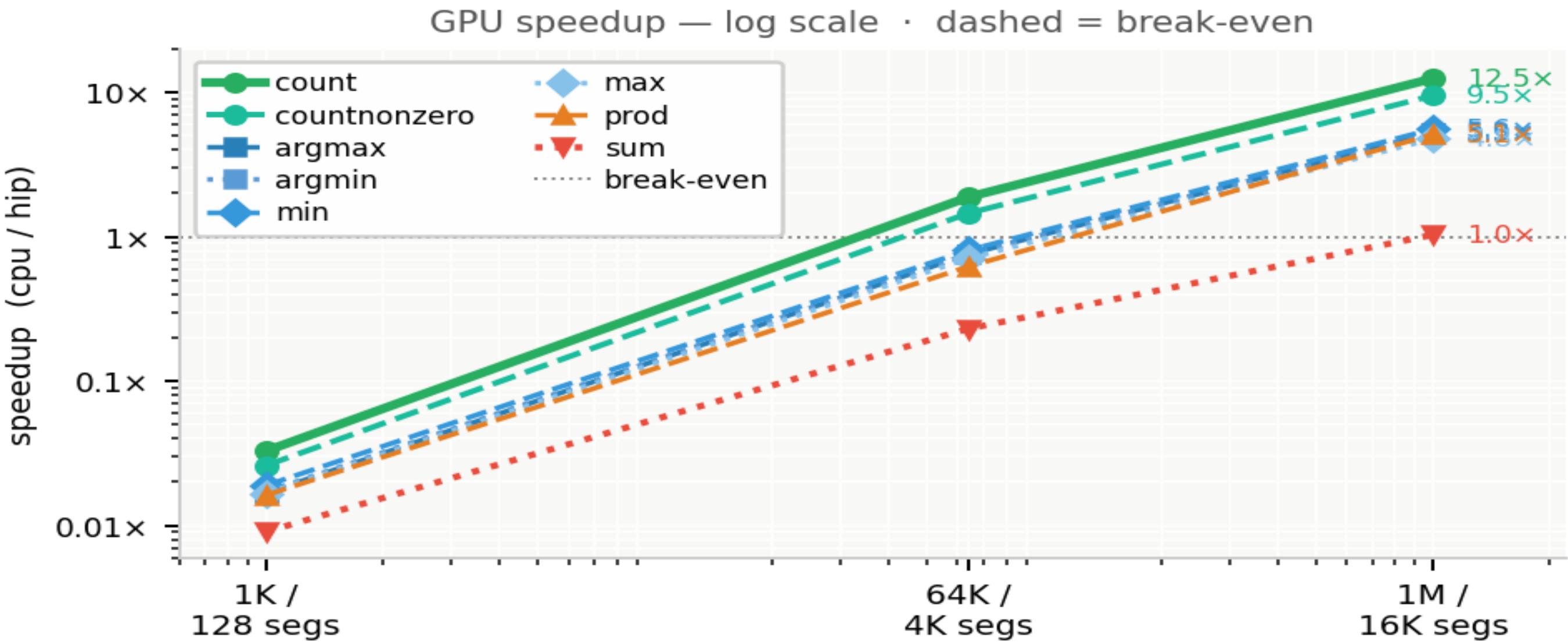
~35 μ s

Best large-size speedup

12.5 $\times$ (count)

kernel tiers · large-size speedup

12.5 $\times$	count write-only, no arithmetic
9.5 $\times$	countnonzero predicated write, still fast
4.8 – 5.6 $\times$	argmax · argmin · max · min · prod compare-reduce, memory-bound
1.03 $\times$	sum AVX2 CPU hits bandwidth parity



Roadmap

K Kernel completeness

Carry, flatten, mask, and sort join reduce — all operations across every numeric type on CPU, Metal, and HIP.

carry

flatten

mask

sort

P Adaptive performance

Profile-guided block and tile selection per kernel per GPU arch — sizes chosen at first run and cached, no hand-tuned constants.

HIP auto-tune

MI300A validated

G Unified GPU backend

Single GpuBackend trait spanning Metal, HIP, and CUDA — one dispatch path, zero per-target boilerplate in kernel code.

Metal

HIP / ROCm

CUDA

E Ecosystem integration

Native columnar interop with RDataFrame and coffea — rawkward arrays usable in HEP pipelines without copy or conversion.

RDataFrame

coffea

MI300A clusters

Takeaway



HIP kernels for irregular data are not trivial

but with the right patterns, they match CUDA and unlock real portability.

the right patterns

01

Loop flattening

Hoist all conditionals out of the inner loop so every lane in the 64-wide wavefront executes the same instruction — zero divergence stalls.

carry · argsort

02

float4 vectorized loads

Pack four 32-bit reads into one 128-bit transaction. Cuts L2 pressure by 4× and saturates HBM2e bandwidth instead of thrashing the cache.

carry · reduce

03

Split over fused kernels

Two lean passes beat one fused pass on HIP. Each split kernel fits the wavefront register file and runs at full occupancy — fusion costs more than it saves.

fusion · argmax · sum

04

Rust macro dispatch

A single match-based macro routes every call to the right vendor backend. Same call site, zero per-target boilerplate — HIP and CUDA diverge only where the hardware demands it.

all kernels